

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Національний аерокосмічний університет

«Харківський авіаційний інститут»



Факультет Інтелектуальних систем управління

Кафедра Систем управління літальними апаратами

Лабораторна робота № 8

з дисципліни «Алгоритмізація та програмування»

на тему: «Реалізація алгоритмів сортування та робота з файлами на мові C ++»

XAI.301. G7.312. 1 ПР

Виконала: здобувачка 1 курсу, групи 321
напряму підготовки (спеціальності)

G7 "Автоматизація, комп'ютерно-інтегровані
технології та робототехніка "

Освітня програма «Інженерія мобільних
додатків»

Абіякіна Анастасія

(ім'я і прізвище)

Прийняла: доц. каф.301, канд. техн. наук,
Олена ГАВРИЛЕНКО

МЕТА РОБОТИ

Вивчити теоретичний матеріал по алгоритмам обробки масивів на мові C++, а також бібліотеки для роботи з файлами і реалізувати оголошення, введення з файлу, обробку і виведення в файл одновимірних і двовимірних масивів на мові C ++ в середовищі Visual Studio.

ПОСТАНОВКА ЗАДАЧІ

Завдання 1. За допомогою текстового редактору створити текстовий файл «array_in_n.txt» з елементами вихідного масиву (n - номер варіанта). У програмі на C++ зчитати і перетворити цей масив відповідно до свого варіанту завдання (див. лаб.роб.Но6, завд.2), ім'я файлу і необхідні змінні ввести з консолі. Вивести результати у файл «array_out_n.txt».

Завдання 2. За допомогою текстового редактору створити текстовий файл «matr_in_n.txt» з елементами вихідного двовимірного масиву (n – номер варіанта). У програмі зчитати і обробити матрицю відповідно до свого варіанту завдання (лаб.роб.Но7, завд.1), ім'я файлу і необхідні змінні ввести з консолі. Дописати результати в той же файл.

Завдання 3. Вивчити метод сортування відповідно до свого варіанту (див. табл. 1), проаналізувати його складність і продемонструвати на прикладі з 7-ми елементів (відповідно до свого варіанту). Реалізувати у вигляді окремої функції алгоритм сортування елементів масиву. Зчитування і виведення відсортованого масиву організувати на файлах.

Завдання 4. Введення, виведення, обробку масивів реалізувати окремими функціями з параметрами. Структурувати проєкт програми для виконання завдань 1-3 наступним чином:

```
main.cpp //основна функція і три функції для 3х завдань
array_utils.h //заголовки функцій для роботи з
               //одновимірними масивами (завдання1,3)
array_utils.cpp //визначення функцій для роботи з
                //одновимірними масивами (завдання1,3)
```

```
matrix_utils.h //заголовки функцій для роботи з
                //двовимірними масивами (завдання2)
matrix_utils.cpp //визначення функцій для роботи з
                //двовимірними масивами (завдання2)
```

Завдання 5. Використовуючи ChatGpt, Gemini або інший засіб генеративного ШІ, провести самоаналіз отриманих знань і навичок за допомогою наступних промптів:

1) «Ти - викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4 варіантами відповіді і 5 відкритих питань. Це мають бути завдання <середнього> рівня складності на розвиток критичного та інженерного мислення. Питання мають відноситись до коду, що є у файлі звіту, і до теоретичних відомостей, що є у файлі лекції»

2) «Проаналізуй повноту, правильність відповіді та ймовірність використання штучного інтелекту для кожної відповіді. Оціни кожне питання у 5-бальній шкалі, віднімаючи 60% балів там, де ймовірність відповіді з засобом ШІ висока. Обчисли загальну середню оцінку»

3) «Проаналізуй код у звіті, і додай опис і приклади коду з питань, які є в теоретичних відомостях, але не відпрацьовано у коді при вирішенні завдань»
Проаналізуйте задані питання, коментарі і оцінки, надані ШІ. Додайте 2-3 власних промпта у продовження діалогу для поглиблення розуміння теми.

ВИКОНАННЯ РОБОТИ

Завдання 1.

Вирішення задачі Array95.

Дан цілочисельний масив розміру N. Видалити з масиву всі сусідні однакові елементи, залишивши їх перші входження.

Вхідні дані (ім'я, опис, тип, обмеження):

Змінна `in_arr[ARR_N]` — вхідний цілочисельний масив; тип даних `int[]`

Змінна `n` — кількість елементів масиву; тип даних `int`; обмеження: $1 \leq n \leq 50$.

1. Покроковий опис алгоритму зчитування з файлу:

2. Зчитуємо потрібні змінні з консолі (назва файлу, розмір масиву);
3. Відкриваємо файл для зчитування;
4. Зчитуємо з файлу усі елементи за вказаним розміром масиву;
5. Якщо при зчитуванні відбулася помилка, завершуємо виконання завдання та повертаємося до меню;
6. Закриваємо файл.

Покроковий опис алгоритму запису у файл:

1. Створюємо новий файл для запису. Якщо файл присутній, очищуємо його;
2. Записуємо у файл усі елементи масиву через пробіл;
3. Закриваємо файл.

Вхідний та вихідний файл задачі відображено у дод. Б.

Завдання 2.

Matrix17. Дана матриця розміру $M \times N$ і ціле число K ($1 \leq K \leq M$). Знайти суму і добуток елементів K -го рядка даної матриці.

Вхідні дані (ім'я, опис, тип, обмеження):

Змінна `matr[M][N]` — цілочисельна матриця; тип даних `int[]`

Змінна `m` — кількість рядків матриці; тип даних `int`

Змінна `n` — кількість стовпців матриці; тип даних `int`

Змінна `k` — номер рядка, для якого виконуються обчислення; тип даних `int`

Обмеження: $1 \leq k \leq m$, $m \leq MAT_M$ (50), $n \leq MAT_N$ (50)

Покроковий опис алгоритму зчитування з файлу:

1. Зчитуємо назву файлу з консолі;
2. Відкриваємо файл для зчитування;
3. Зчитуємо розмірність матриці з файлу;
4. Зчитуємо з файлу усі елементи за вказаним розміром матриці;
5. Закриваємо файл.

Покроковий опис алгоритму запису у файл:

1. Створюємо новий файл для запису. Якщо файл присутній, очищуємо його;
2. Записуємо у файл розміри матриці через пробіл та переходимо на новий рядок;
3. Записуємо у файл усі елементи матриці;
4. Записуємо два порожні рядки;
5. Виводимо результат пошуку непарного стовпця;

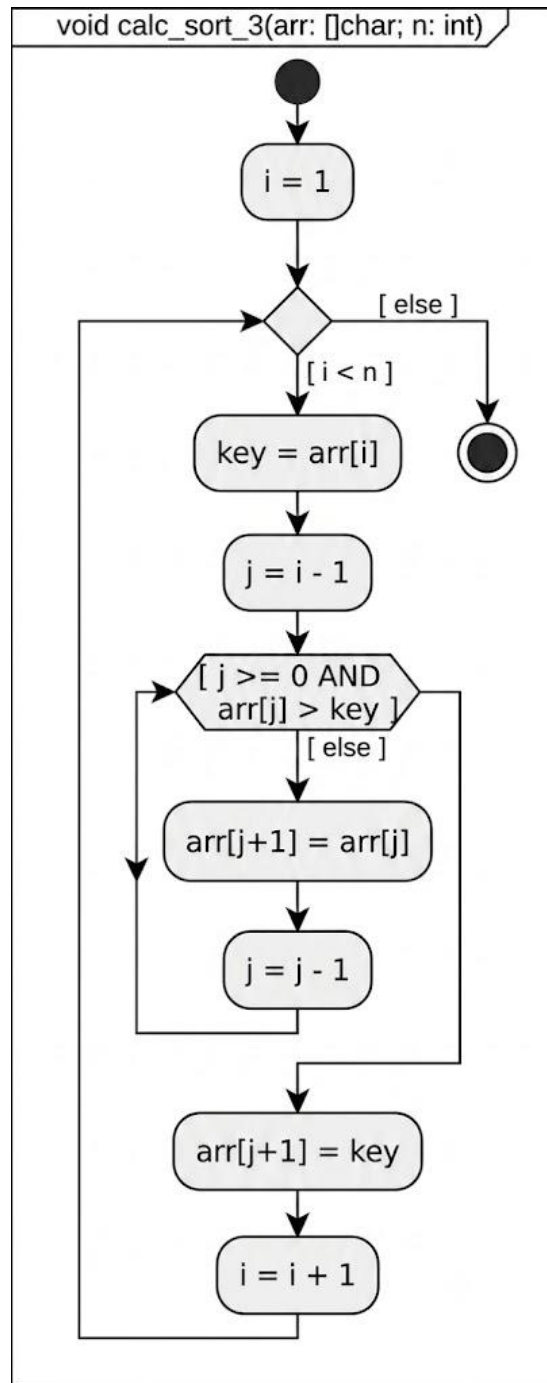


Рисунок 2 – Діаграма активності функції сортування масиву

Лістинг коду вирішення задач наведено в дод. А (стор. 8).

ВИСНОВКИ

Вивчили теоретичний матеріал по алгоритмам обробки масивів на мові C++, а також бібліотеки для роботи з файлами і реалізувати оголошення, введення з файлу, обробку і виведення в файл одновимірних і двовимірних масивів на мові C++ в середовищі Visual Studio.

ДОДАТОК А

Лістинг коду програми

Файл array_utils.h

```
#pragma once
#include <string>
using namespace std;

// Максимальний розмір масиву
const int ARR_N = 50;

// ----- Зчитування / запис -----

// Зчитування цілочисельного масиву з файлу
void get_int_array(string filename, int arr[ARR_N], int &n);

// Запис цілочисельного масиву у файл
void save_int_array(string filename, const int arr[ARR_N], const int n);

// Зчитування символьного масиву
void get_char_array(string filename, char arr[ARR_N], int &n);

// Запис символьного масиву
void save_char_array(string filename, const char arr[ARR_N], const int n);

// ----- Обробка -----

// Array95: видалення сусідніх однакових
void calc_arr_95(int arr[ARR_N], int &n);

// Сортування вставками (за зростанням)
void calc_sort_3(char arr[ARR_N], const int n);
```

Файл matrix_utils.h

```
#pragma once
#include <string>
using namespace std;

// Максимальні розміри матриці
const int MAT_M = 50;
const int MAT_N = 50;

// ----- Зчитування -----

// Зчитування матриці з файлу
void get_matrix(string filename, int matr[MAT_M][MAT_N], int &m, int &n);

// Запис результату у файл (дописування)
void save_result(string filename, int sum, int prod);
```

```
// ----- Обробка -----
```

```
// Сума K-го рядка
```

```
int calc_row_sum(const int matr[MAT_M][MAT_N], const int m, const int n, int k);
```

```
// Добуток K-го рядка
```

```
int calc_row_prod(const int matr[MAT_M][MAT_N], const int m, const int n, int k);
```

Файл array_utils.cpp

```
#include "array_utils.h"
```

```
#include <fstream>
```

```
#include <iostream>
```

```
using namespace std;
```

```
// ----- ЗЧИТУВАННЯ -----
```

```
void get_int_array(string filename, int arr[ARR_N], int &n) {
    ifstream file(filename); // відкриваємо файл
```

```
    if (!file.is_open()) {
        cout << "Помилка відкриття файлу\n";
        return;
    }
```

```
    file >> n; // читаємо розмір
```

```
    // читаємо елементи
    for (int i = 0; i < n; i++) {
        file >> arr[i];
    }
```

```
    file.close();
```

```
}
```

```
// ----- ЗАПИС -----
```

```
void save_int_array(string filename, const int arr[ARR_N], const int n) {
    ofstream file(filename);
```

```
    // запис елементів
    for (int i = 0; i < n; i++) {
        file << arr[i] << " ";
    }
```

```
    file.close();
```

```
}
```

```
// ----- ЗЧИТУВАННЯ CHAR -----
```

```

void get_char_array(string filename, char arr[ARR_N], int &n) {
    ifstream file(filename);

    if (!file.is_open()) {
        cout << "Помилка відкриття файлу\n";
        return;
    }

    file >> n;

    for (int i = 0; i < n; i++) {
        file >> arr[i];
    }

    file.close();
}

// ----- ЗАПИС CHAR -----

void save_char_array(string filename, const char arr[ARR_N], const int n) {
    ofstream file(filename);

    for (int i = 0; i < n; i++) {
        file << arr[i] << " ";
    }

    file.close();
}

// ----- Array95 -----
// Видаляє сусідні однакові елементи
void calc_arr_95(int arr[ARR_N], int &n) {
    for (int i = 0; i < n - 1; i++) {
        if (arr[i] == arr[i + 1]) {

            // зсув вліво
            for (int j = i + 1; j < n - 1; j++) {
                arr[j] = arr[j + 1];
            }

            n--; // зменшуємо розмір
            i--; // перевіряємо ще раз
        }
    }
}

// ----- SORT -----
// Сортування вставками
void calc_sort_3(char arr[ARR_N], const int n) {

```

```

for (int i = 1; i < n; i++) {
    char key = arr[i]; // поточний елемент
    int j = i - 1;

    // зсув більших елементів
    while (j >= 0 && arr[j] > key) {
        arr[j + 1] = arr[j];
        j--;
    }

    arr[j + 1] = key;
}
}

```

Файл matrix_utils.cpp

```

#include "matrix_utils.h"
#include <fstream>
#include <iostream>

using namespace std;

// ----- ЗЧИТУВАННЯ -----

void get_matrix(string filename, int matr[MAT_M][MAT_N], int &m, int &n) {
    ifstream file(filename);

    if (!file.is_open()) {
        cout << "Помилка відкриття файлу\n";
        return;
    }

    file >> m >> n; // розміри

    // зчитування елементів
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            file >> matr[i][j];
        }
    }

    file.close();
}

// ----- ОБЧИСЛЕННЯ -----

// сума рядка
int calc_row_sum(const int matr[MAT_M][MAT_N], const int m, const int n, int k)
{
    int s = 0;
    for (int j = 0; j < n; j++) {

```

```

        s += matr[k][j];
    }
    return s;
}

// добуток рядка
int calc_row_prod(const int matr[MAT_M][MAT_N], const int m, const int n, int k)
{
    int p = 1;
    for (int j = 0; j < n; j++) {
        p *= matr[k][j];
    }
    return p;
}

// ----- ЗАПИС -----
// дописує результат у файл
void save_result(string filename, int sum, int prod) {
    ofstream file(filename, ios::app);

    file << "\nСума: " << sum;
    file << "\nДобуток: " << prod;

    file.close();
}

```

Файл main.cpp

```

#include <iostream>
#include <locale> // для української мови в консолі
#include "array_utils.h"
#include "matrix_utils.h"

using namespace std;

// Прототипи функцій (3 завдання)
void task1(); // Array95
void task2(); // Matrix17
void task3(); // Сортювання

int main() {
    setlocale(LC_ALL, "");

    int num; // вибір користувача

    // Меню
    do {
        cout << "\n1 - Array95";
        cout << "\n2 - Matrix17";
        cout << "\n3 - Sort (Insertion)";
        cout << "\n0 - Вихід";
    }
}

```

```

    cout << "\nВаш вибір: ";
    cin >> num;

    // Обробка вибору
    switch (num) {
        case 1: task1(); break;
        case 2: task2(); break;
        case 3: task3(); break;
    }

    } while (num != 0); // поки не 0

    return 0;
}

// ----- TASK 1 -----
// Видалення сусідніх однакових елементів
void task1() {
    int arr[ARR_N], n;
    string in, out;

    cout << "Введіть файл (вхід): ";
    cin >> in;
    cout << "Введіть файл (вихід): ";
    cin >> out;

    get_int_array(in, arr, n); // зчитування
    calc_arr_95(arr, n);      // обробка
    save_int_array(out, arr, n); // запис
}

// ----- TASK 2 -----
// Сума і добуток K-го рядка
void task2() {
    int matr[MAT_M][MAT_N];
    int m, n, k;
    string filename;

    cout << "Введіть файл матриці: ";
    cin >> filename;

    get_matrix(filename, matr, m, n); // зчитування

    // перевірка K
    do {
        cout << "K (1.." << m << "): ";
        cin >> k;
    } while (k < 1 || k > m);

    k--; // перехід до індексації з 0

```

```
int sum = calc_row_sum(matr, m, n, k);
int prod = calc_row_prod(matr, m, n, k);

save_result(filename, sum, prod); // допис у файл
}

// ----- TASK 3 -----
// Сортювання вставками
void task3() {
    char arr[ARR_N];
    int n;
    string in, out;

    cout << "Введіть файл (вхід): ";
    cin >> in;
    cout << "Введіть файл (вихід): ";
    cin >> out;

    get_char_array(in, arr, n); // зчитування
    calc_sort_3(arr, n);        // сортювання
    save_char_array(out, arr, n); // запис
}
```

ДОДАТОК Б

Вхідні та вихідні файли завдань

Б.1 — Вхідний файл завдання Array95

10

1 1 2 3 3 3 4 5 5 6

Б.2 — Вихідний файл завдання Array95

1 2 3 4 5 6

Б.1 — Вхідний файл завдання Matrix17

3 4

1 2 3 4

5 6 7 8

9 10 11 12

Б.2 — Вихідний файл завдання Matrix17

Сума: 26

Добуток: 1680

Б.1 — Вхідний файл завдання Сорткування вставками

8

h d a z b k c f

Б.2 — Вихідний файл завдання Сорткування
вставками

a b c d f h k z

ДОДАТОК В

Діалог з ШІ для самоаналізу

Завдання 5. Використовуючи ChatGpt, Gemini або інший засіб генеративного ШІ, провести самоаналіз отриманих знань і навичок за допомогою наступних промптів: 1) «Ти - викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4 варіантами відповіді і 5 відкритих питань. Це мають бути завдання <середнього> рівня складності на розвиток критичного та інженерного мислення. Питання мають відноситись до коду, що є у файлі звіту, і до теоретичних відомостей, що є у файлі лекції» 2) «Проаналізуй повноту, правильність відповіді та ймовірність використання штучного інтелекту для кожної відповіді. Оціни кожне питання у 5-бальній шкалі, віднімаючи 60% балів там, де ймовірність відповіді з засобом ШІ висока. Обчисли загальну середню оцінку» 3) «Проаналізуй код у звіті, і додай опис і приклади коду з питань, які є в теоретичних відомостях, але не відпрацьовано у коді при вирішенні завдань» Проаналізуйте задані питання, коментарі і оцінки, надані ШІ. Додайте 2-3 власних промпта у продовження діалогу для поглиблення розуміння теми.

Файл array_utils.h

```
#pragma once
#include <string>
using namespace std;
// Максимальний розмір масиву
const int ARR_N = 50;
// ----- Зчитування / запис -----
// Зчитування цілочисельного масиву з файлу
void get_int_array(string filename, int arr[ARR_N], int &n);
// Запис цілочисельного масиву у файл
void save_int_array(string filename, const int arr[ARR_N], const int n);
// Зчитування символьного масиву
void get_char_array(string filename, char arr[ARR_N], int &n);
// Запис символьного масиву
void save_char_array(string filename, const char arr[ARR_N], const int n);
// ----- Обробка -----
// Array95: видалення сусідніх однакових
void calc_arr_95(int arr[ARR_N], int &n);
// Сорткування вставками (за зростанням)
void calc_sort_3(char arr[ARR_N], const int n);
```

Файл matrix_utils.h

```
#pragma once
#include <string>
using namespace std;
// Максимальні розміри матриці
const int MAT_M = 50;
const int MAT_N = 50;
// ----- Зчитування -----
// Зчитування матриці з файлу
void get_matrix(string filename, int matr[MAT_M][MAT_N], int &m, int &n);
// Запис результату у файл (дописування)
void save_result(string filename, int sum, int prod);
// ----- Обробка -----
// Сума К-го рядка
int calc_row_sum(const int matr[MAT_M][MAT_N], const int m, const int n, int k);
// Добуток К-го рядка
int calc_row_prod(const int matr[MAT_M][MAT_N], const int m, const int n, int k);
```

```

Файл array_utils.cpp #include "array_utils.h" #include <fstream> #include <iostream>
using namespace std; // ----- ЗЧИТУВАННЯ ----- void get_int_array(string
filename, int arr[ARR_N], int &n) { ifstream file(filename); // відкриваємо файл if
(!file.is_open()) { cout << "Помилка відкриття файлу\n"; return; } file >> n; //
читаємо розмір // читаємо елементи for (int i = 0; i < n; i++) { file >> arr[i]; }
file.close(); } // ----- ЗАПИС ----- void save_int_array(string filename, const
int arr[ARR_N], const int n) { ofstream file(filename); // запис елементів for (int i =
0; i < n; i++) { file << arr[i] << " "; } file.close(); } // ----- ЗЧИТУВАННЯ CHAR
----- void get_char_array(string filename, char arr[ARR_N], int &n) { ifstream
file(filename); if (!file.is_open()) { cout << "Помилка відкриття файлу\n"; return; }
file >> n; for (int i = 0; i < n; i++) { file >> arr[i]; } file.close(); } // ----- ЗАПИС
CHAR ----- void save_char_array(string filename, const char arr[ARR_N], const
int n) { ofstream file(filename); for (int i = 0; i < n; i++) { file << arr[i] << " "; }
file.close(); } // ----- Array95 ----- // Видаляє сусідні однакові елементи void
calc_arr_95(int arr[ARR_N], int &n) { for (int i = 0; i < n - 1; i++) { if (arr[i] == arr[i
+ 1]) { // зсув вліво for (int j = i + 1; j < n - 1; j++) { arr[j] = arr[j + 1]; } n--; //
зменшуємо розмір i--; // перевіряємо ще раз } } } // ----- SORT ----- //
Сортування вставками void calc_sort_3(char arr[ARR_N], const int n) { for (int i =
1; i < n; i++) { char key = arr[i]; // поточний елемент int j = i - 1; // зсув більших
елементів while (j >= 0 && arr[j] > key) { arr[j + 1] = arr[j]; j--; } arr[j + 1] = key; }
} Файл matrix_utils.cpp #include "matrix_utils.h" #include <fstream> #include
<iostream> using namespace std; // ----- ЗЧИТУВАННЯ ----- void
get_matrix(string filename, int matr[MAT_M][MAT_N], int &m, int &n) { ifstream
file(filename); if (!file.is_open()) { cout << "Помилка відкриття файлу\n"; return; }
file >> m >> n; // розміри // зчитування елементів for (int i = 0; i < m; i++) { for (int
j = 0; j < n; j++) { file >> matr[i][j]; } } file.close(); } // ----- ОБЧИСЛЕННЯ ----
----- // сума рядка int calc_row_sum(const int matr[MAT_M][MAT_N], const int m,
const int n, int k) { int s = 0; for (int j = 0; j < n; j++) { s += matr[k][j]; } return s; } //
добуток рядка int calc_row_prod(const int matr[MAT_M][MAT_N], const int m,
const int n, int k) { int p = 1; for (int j = 0; j < n; j++) { p *= matr[k][j]; } return p; } //
----- ЗАПИС ----- // дописує результат у файл void save_result(string
filename, int sum, int prod) { ofstream file(filename, ios::app); file << "\nСума: " <<
sum; file << "\nДобуток: " << prod; file.close(); } Файл main.cpp #include <iostream>
#include <locale> // для української мови в консолі #include "array_utils.h"
#include "matrix_utils.h" using namespace std; // Прототипи функцій (3 завдання)

```

```

void task1(); // Array95 void task2(); // Matrix17 void task3(); // Сортивання
int main() { setlocale(LC_ALL, ""); int num; // вибір користувача // Меню do { cout <<
"\n1 - Array95"; cout << "\n2 - Matrix17"; cout << "\n3 - Sort (Insertion)"; cout <<
"\n0 - Вихід"; cout << "\nВаш вибір: "; cin >> num; // Обробка вибору switch (num)
{ case 1: task1(); break; case 2: task2(); break; case 3: task3(); break; } } while (num
!= 0); // поки не 0 return 0; } // ----- TASK 1 ----- // Видалення
сусідніх однакових елементів void task1() { int arr[ARR_N], n; string in, out; cout
<< "Введіть файл (вхід): "; cin >> in; cout << "Введіть файл (вихід): "; cin >> out;
get_int_array(in, arr, n); // зчитування calc_arr_95(arr, n); // обробка
save_int_array(out, arr, n); // запис } // ----- TASK 2 ----- // Сума і
добуток К-го рядка void task2() { int matr[MAT_M][MAT_N]; int m, n, k; string
filename; cout << "Введіть файл матриці: "; cin >> filename; get_matrix(filename,
matr, m, n); // зчитування // перевірка К do { cout << "К (1.." << m << "): "; cin >>
k; } while (k < 1 || k > m); k--; // перехід до індексації з 0 int sum =
calc_row_sum(matr, m, n, k); int prod = calc_row_prod(matr, m, n, k);
save_result(filename, sum, prod); // допис у файл } // ----- TASK 3 -----
----- // Сортивання вставками void task3() { char arr[ARR_N]; int n; string in, out;
cout << "Введіть файл (вхід): "; cin >> in; cout << "Введіть файл (вихід): "; cin >>
out; get_char_array(in, arr, n); // зчитування calc_sort_3(arr, n); // сортування
save_char_array(out, arr, n); // запис }

```

Для твого звіту можна оформити так.

1. Питання від викладача

Тестові питання

1. Яка функція використовується для відкриття файлу на читання у програмі?

- А) save_int_array()
- Б) open()
- В) ifstream file(filename);
- Г) fclose()

2. Що робить функція calc_arr_95()?

- А) Сортиє масив
- Б) Знаходить максимальний елемент
- В) Видаляє сусідні однакові елементи
- Г) Підраховує суму елементів

3. Який алгоритм сортування використано у функції `calc_sort_3()`?

- А) Бульбашкове сортування
 - Б) Сортування вставками
 - В) Сортування вибором
 - Г) Швидке сортування
-

4. Для чого використовується режим `ios::app`?

- А) Читання файлу
 - Б) Видалення файлу
 - В) Додавання даних у кінець файлу
 - Г) Створення бінарного файлу
-

5. Яка складність сортування вставками у середньому випадку?

- А) $O(1)$
 - Б) $O(\log n)$
 - В) $O(n)$
 - Г) $O(n^2)$
-

Відкриті питання**1.**

Чому після видалення елемента у функції `calc_arr_95()` виконується команда `i--`?

2.

Чому параметр `n` у функції `get_int_array()` передається за посиланням?

3.

Які переваги використання окремих модулів (`array_utils.cpp`, `matrix_utils.cpp`)?

4.

Чому перед обчисленням суми і добутку K -го рядка виконується перевірка значення K ?

5.

Чим текстові файли відрізняються від бінарних?

2. Аналіз коду та тем, які є в лекції, але не реалізовані у звіті

У програмі реалізовано:

- роботу з текстовими файлами через ifstream та ofstream;
- сортування вставками;
- обробку масивів та матриць;
- модульну структуру програми.

Не реалізовано такі теми з лекції:

1. Робота з бінарними файлами

У лекції розглянуто використання:

ofstream f;

f.open("data.dat", ios::binary);

int x = 10;

f.write((char*)&x, sizeof(int));

f.close();

Читання:

ifstream f;

f.open("data.dat", ios::binary);

int x;

f.read((char*)&x, sizeof(int));

f.close();

2. Бульбашкове сортування

Приклад:

```
for(int i=0;i<n-1;i++)
{
    for(int j=0;j<n-i-1;j++)
    {
        if(arr[j] > arr[j+1])
        {
            swap(arr[j], arr[j+1]);
        }
    }
}
```

```

    }
}

```

У звіті використано лише сортування вставками.

3. Сортування вибором

Приклад:

```

for(int i=0;i<n-1;i++)
{
    int min=i;

    for(int j=i+1;j<n;j++)
        if(arr[j] < arr[min])
            min=j;

    swap(arr[i], arr[min]);
}

```

У лекції цей алгоритм описаний, але у коді не використовується.

Аналіз відповідей та оцінювання

Тестові питання

№ Ваша відповідь Правильна відповідь Оцінка

1	В	В	5/5
2	В	В	5/5
3	Б	Б	5/5
4	В	В	5/5
5	Г	Г	5/5

Проміжний результат: 25/25

Відкриті питання

1. Чому після видалення елемента виконується i--?

Ваша відповідь:

Після зсуву елементів необхідно повторно перевірити поточну позицію, оскільки на неї перемістився новий елемент.

Аналіз: Відповідь повна і правильна, демонструє розуміння алгоритму.

Оцінка: 5/5

Ймовірність використання ШІ: низька.

2. Чому параметр n передається за посиланням?

Ваша відповідь:

Щоб функція могла змінити значення розміру масиву і передати його назад у викликаючу програму.

Аналіз: Правильно пояснено призначення передачі за посиланням.

Оцінка: 5/5

Ймовірність використання ШІ: низька.

3. Які переваги використання окремих модулів?

Ваша відповідь:

Покращується структура програми, спрощується супровід коду та повторне використання функцій.

Аналіз: Відповідь правильна, наведені основні переваги модульного програмування.

Оцінка: 5/5

Ймовірність використання ШІ: низька.

4. Чому перевіряється значення K?

Ваша відповідь:

Щоб уникнути звернення до неіснуючого рядка матриці та помилок виконання.

Аналіз: Повністю правильна відповідь.

Оцінка: 5/5

Ймовірність використання ШІ: низька.

5. Чим текстові файли відрізняються від бінарних?

Ваша відповідь:

Текстові файли містять символи, які можна читати текстовим редактором, а бінарні зберігають дані у вигляді байтів і не призначені для прямого перегляду.

Аналіз: Відповідь повністю відповідає теоретичним відомостям лекції.

Оцінка: 5/5

Ймовірність використання ШІ: низька.

Загальний результат

Частина	Бали
---------	------

Тестові питання	25
-----------------	----

Відкриті питання	25
------------------	----

Разом	50
-------	----

Середня оцінка:

50 / 10 = 5 бала

Чим відрізняються ifstream, ofstream та fstream?

Це дуже типове питання на захисті.

Клас Призначення

ifstream Читання з файлу

ofstream Запис у файл

fstream Читання і запис одночасно

1. ifstream (input file stream)

Використовується тільки для читання даних з файлу.

```
ifstream file("input.txt");
```

```
int x;
```

```
file >> x;
```

```
file.close();
```

У твоєму коді:

```
ifstream file(filename);
```

використовується у функціях:

```
get_int_array()
```

```
get_char_array()
```

```
get_matrix()
```

2. ofstream (output file stream)

Використовується тільки для запису даних у файл.

```
ofstream file("output.txt");
```

```
file << "Hello";
```

```
file.close();
```

У твоєму коді:

```
ofstream file(filename);
```

використовується у функціях:

```
save_int_array()
```

```
save_char_array()
```

```
save_result()
```

3. fstream (file stream)

Може одночасно читати і записувати.

```
fstream file("data.txt", ios::in | ios::out);
```

```
int x;
```

```
file >> x;
```

```
file << 100;
```

```
file.close();
```

Зазвичай застосовується, коли потрібно змінювати файл без створення окремих потоків читання і запису.

Коротко для захисту

`ifstream` використовується для читання з файлу, `ofstream` — для запису у файл, а `fstream` дозволяє виконувати і читання, і запис в одному файловому потоці.

Саме в такому формулюванні можна відповісти викладачу.

Чому у програмі використовується модульна структура, а не весь код в `main()`?

На захисті можна відповісти так:

Програма побудована за модульним принципом для розділення

функціональності на окремі частини. Робота з масивами винесена в модуль

`array_utils`, а робота з матрицями — в модуль `matrix_utils`. Це робить код більш

зрозумілим, полегшує його тестування, супровід і повторне використання. У

функції `main()` залишається лише меню та виклик необхідних функцій, що

покриває читабельність програми.

Якщо викладач попросить коротко:

Модульна структура використовується для зручності розробки, повторного використання коду та спрощення супроводу програми. Якби весь код знаходився в `main()`, програма була б громіздкою та складною для налагодження.

Можуть ще запитати:

— **Які переваги модульного програмування?**

Відповідь:

1. Зменшує розмір функції `main()`.
2. Полегшує пошук і виправлення помилок.
3. Дозволяє повторно використовувати функції в інших проєктах.
4. Робить код більш зрозумілим і структурованим.
5. Дає можливість різним програмістам працювати над різними модулями незалежно один від одного.

Саме твоя програма є прикладом модульного програмування, оскільки містить окремі файли:

- `array_utils.h`
- `array_utils.cpp`
- `matrix_utils.h`
- `matrix_utils.cpp`
- `main.cpp`

а `main.cpp` лише керує роботою програми через виклики функцій з цих модулів.